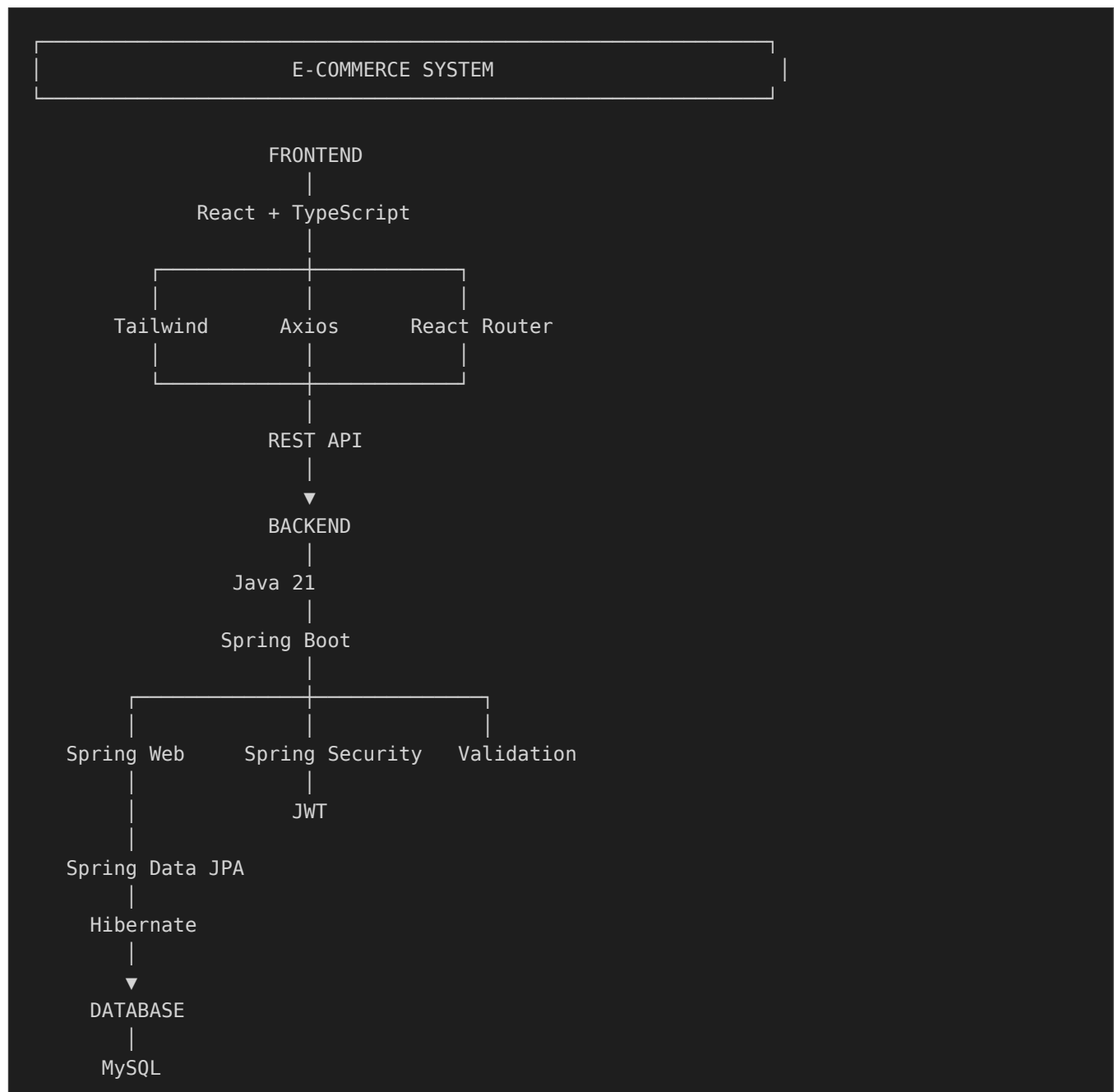


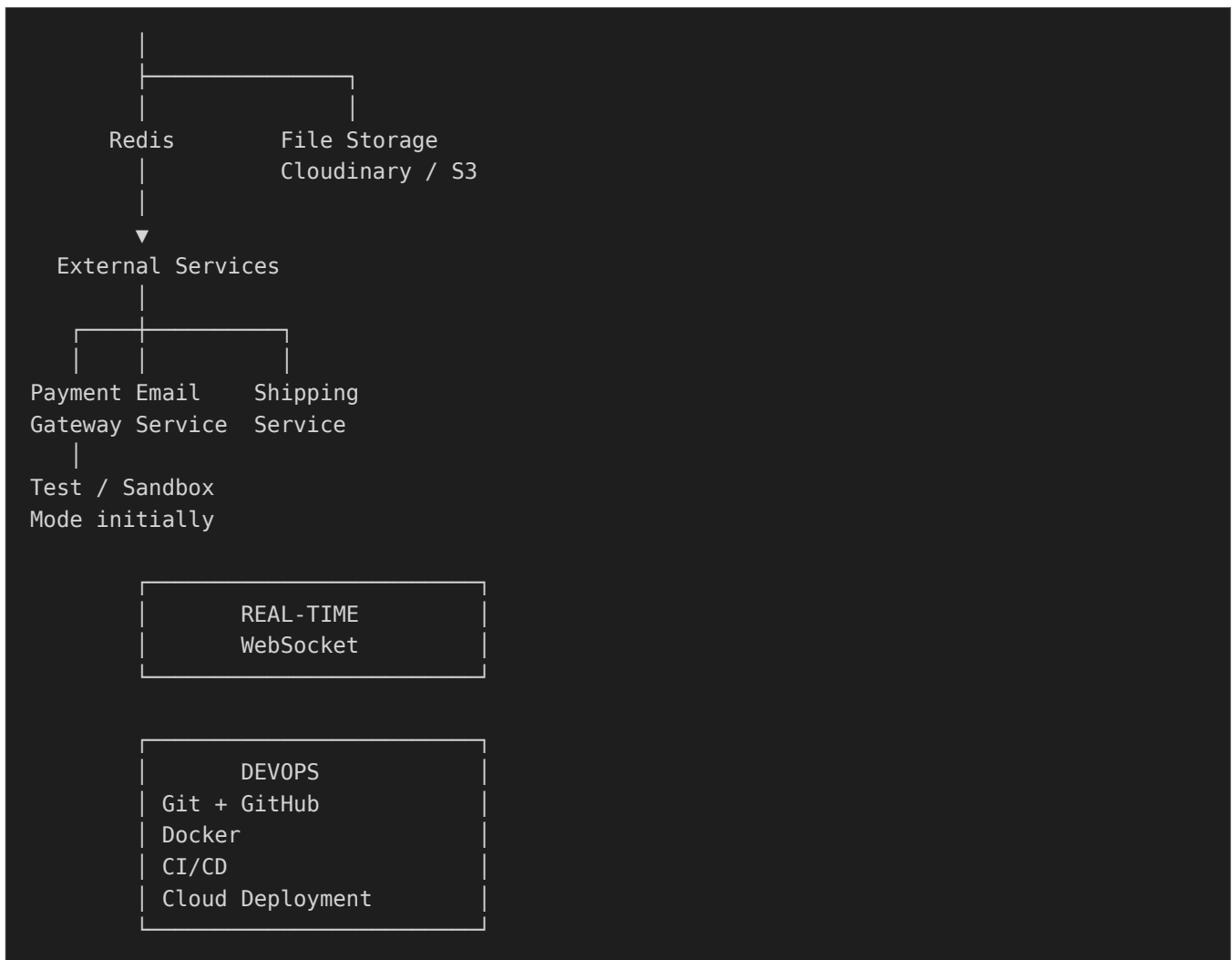
E-Commerce Application

Complete Technology Stack & Project Architecture

1. Complete Technology Stack — Overview

For a production-style e-commerce application, the system is organised into distinct layers: frontend, backend, database, external services, real-time communication, and DevOps.





2. Frontend Technology

Technology	Purpose
React	Builds the UI
TypeScript	Type-safe JavaScript
React Router	Client-side navigation between pages
Axios	Communicates with the Spring Boot API
Tailwind CSS	Styling
Redux Toolkit / Context API	State management (user, cart, wishlist, auth)

Routes

```
/
/products
/products/10
/cart
/checkout
/orders
/profile
/admin
```

Redux is not required from day one — state management can start with the simpler Context API and be upgraded to Redux Toolkit later if complexity grows.

3. Backend Technology

```
Java 21
|
Spring Boot
|
├─ Spring Web
├─ Spring Security
├─ Spring Data JPA
├─ Hibernate
├─ Bean Validation
└─ Maven
```

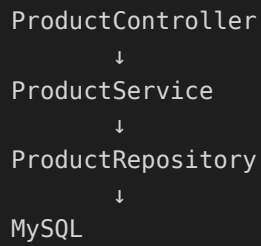
Request Flow

This is one of the most important concepts to understand: every request travels through a consistent set of layers before reaching the database.

```
React
|
| HTTP Request
▼
Controller
|
▼
Service
|
▼
Repository
|
▼
JPA / Hibernate
```

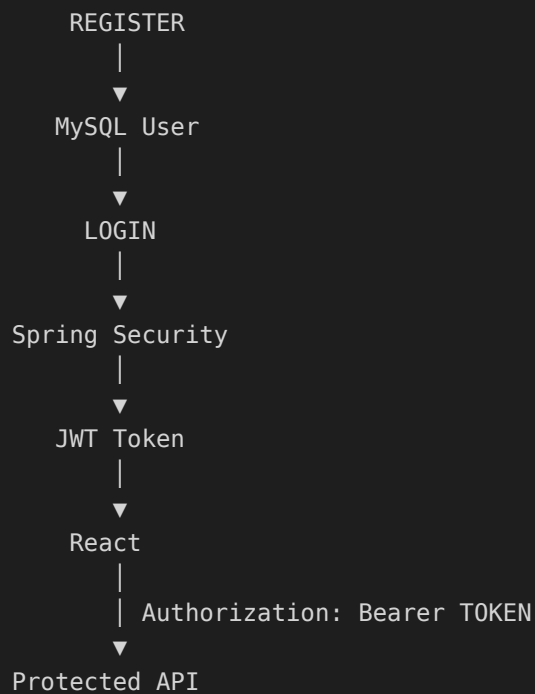


Example — POST /api/products:



4. Authentication Flow

Authentication is handled with Spring Security combined with JWT.



Roles

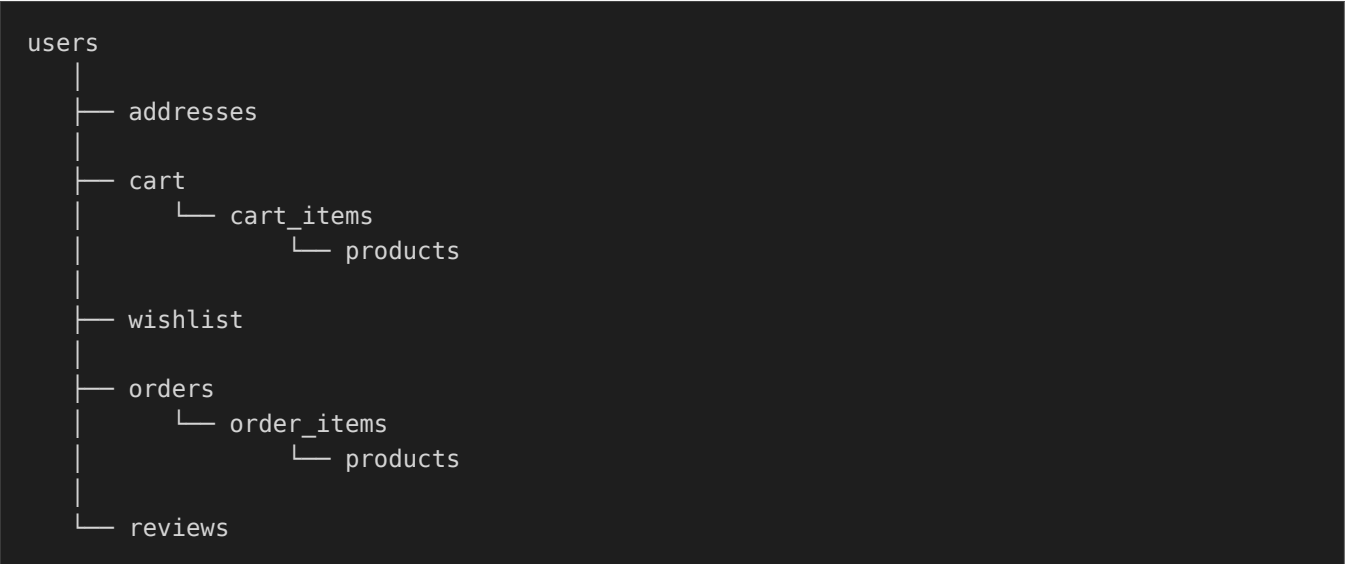
Role	Permissions
CUSTOMER	View products · Add to cart · Place orders · View own orders

Role	Permissions
ADMIN	Add / update / delete products · Manage orders · Manage users

5. Database

The system uses MySQL as the primary relational database.

Core Entity Structure



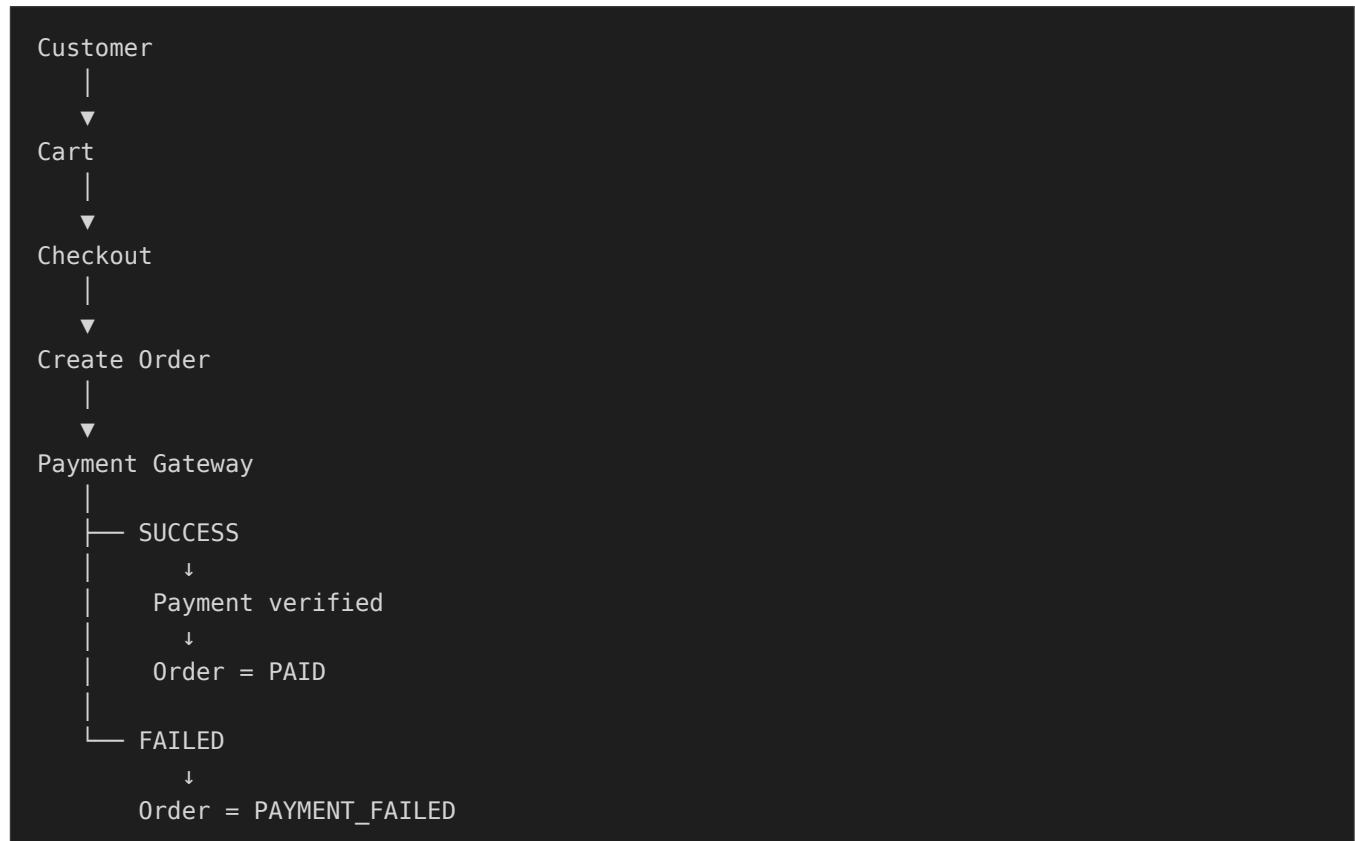
Full Table List

users
roles
addresses
categories
products
product_images
cart
cart_items
wishlist
wishlist_items
orders
order_items
payments
reviews
coupons

Exact relationships (foreign keys, cardinality) will be finalised at the database design stage.

6. Payment Flow

Development starts with test / sandbox payment mode only.



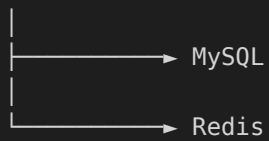
Path to Production



7. Redis

Redis is introduced later, only once a concrete use case exists — not added purely because it is popular.

Spring Boot



Possible Uses

- Product caching
- Session / temporary data
- Rate limiting
- Frequently accessed information

8. Real-Time System

```
React
|
| WebSocket
▼
Spring Boot
|
▼
Order / Event System
```

Example:

```
ADMIN
|
| Changes order
|
▼
Spring Boot
|
| WebSocket event
▼
CUSTOMER
|
▼
"Your order has been shipped"
```

No page refresh is required.

9. Image / File Storage

Product images are not stored as binary data directly in MySQL.

```
React
|
▼
Spring Boot
|
▼
Cloudinary / S3
|
▼
Image URL
|
▼
MySQL
```

MySQL only stores a reference:

```
product_id
image_url
```

The actual image file lives in object / file storage (Cloudinary or S3).

10. Email

Used for:

- Registration
- Password reset
- Order confirmation
- Payment confirmation
- Shipping notification

```
Spring Boot
|
▼
Email Service
|
▼
Customer's Email
```

This layer can be added later in the build.

11. DevOps

```
Git
↓
GitHub
↓
Docker
↓
CI/CD
↓
Cloud
```

Docker Structure (eventual)

```
Docker
|
├─ React container
├─ Spring Boot container
├─ MySQL container
└─ Redis container
```

Docker makes the local development environment much easier to reproduce consistently across machines.

12. Complete Project Tree

The project is structured as two top-level applications:

```
ecommerce/
|
├─ frontend/
|
└─ backend/
```

Frontend Structure

```
frontend/
|
├─ public/
|
└─ src/
```

```
|
|
|— assets/
|
|— components/
|   |— Navbar/
|   |— Footer/
|   |— ProductCard/
|   |— Button/
|   |— Modal/
|
|— pages/
|   |— Home/
|   |— Products/
|   |— ProductDetails/
|   |— Login/
|   |— Register/
|   |— Cart/
|   |— Checkout/
|   |— Orders/
|   |— Profile/
|   |— Admin/
|
|— services/
|   |— api.ts
|   |— authApi.ts
|   |— productApi.ts
|   |— cartApi.ts
|   |— orderApi.ts
|
|— context/
|   |— AuthContext.tsx
|   |— CartContext.tsx
|
|— hooks/
|— types/
|— utils/
|— App.tsx
|— main.tsx
|
|— package.json
|— tsconfig.json
```

Backend Structure

```
backend/
|
|— src/
|   |
|   |— main/
```

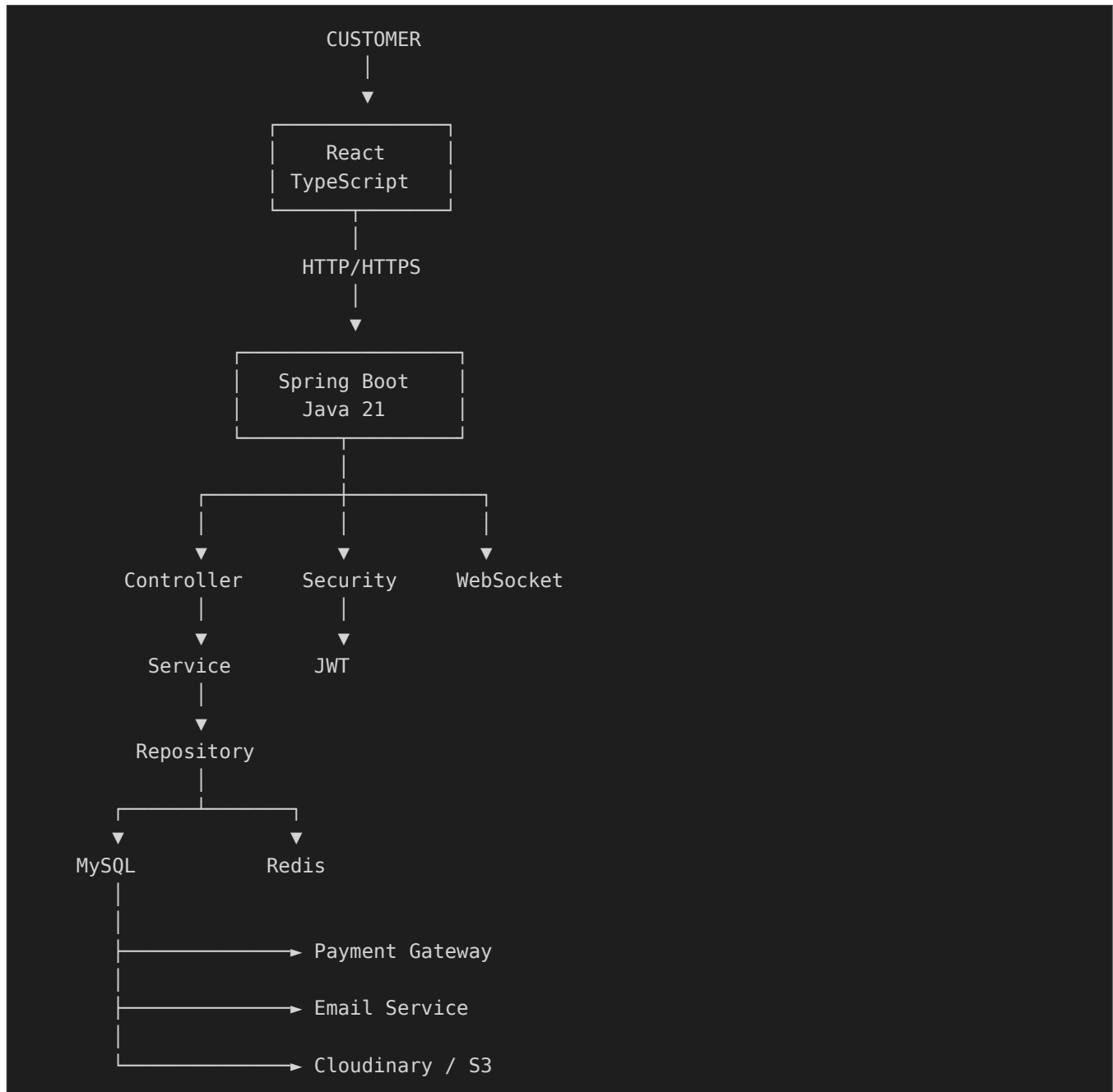
```

├── java/
│   ├── com/example/ecommerce/
│   │   ├── EcommerceApplication.java
│   │   ├── config/
│   │   │   ├── SecurityConfig.java
│   │   │   └── CorsConfig.java
│   │   ├── auth/
│   │   │   ├── controller/
│   │   │   ├── service/
│   │   │   ├── dto/
│   │   │   └── security/
│   │   ├── user/
│   │   │   ├── controller/
│   │   │   ├── service/
│   │   │   ├── repository/
│   │   │   ├── entity/
│   │   │   └── dto/
│   │   ├── product/
│   │   │   ├── controller/
│   │   │   ├── service/
│   │   │   ├── repository/
│   │   │   ├── entity/
│   │   │   └── dto/
│   │   ├── category/
│   │   ├── cart/
│   │   ├── wishlist/
│   │   ├── order/
│   │   ├── payment/
│   │   ├── review/
│   │   ├── coupon/
│   │   ├── address/
│   │   ├── notification/
│   │   └── common/
│   └── resources/
│       ├── application.properties
│       └── static/
├── test/
├── pom.xml
└── Dockerfile

```

13. How Everything Communicates

The big picture to keep in mind throughout development:



14. Development Roadmap

The system is not built all at once — it is delivered in phases, each adding a working, testable increment.

Phase 1

```
Java 21
↓
Spring Boot
↓
MySQL
↓
REST API
↓
Product CRUD
```

Phase 2

```
User Registration
↓
Login
↓
JWT
↓
Spring Security
↓
Roles
```

Phase 3

```
React
↓
Connect APIs
↓
Products
↓
Product Details
```

Phase 4

```
Cart
↓
Wishlist
↓
Address
↓
Checkout
```

Phase 5

```
Orders
  ↓
Test Payment Gateway
  ↓
Payment Verification
```

Phase 6

```
Admin Dashboard
  ↓
Product Management
  ↓
Order Management
  ↓
User Management
```

Phase 7

```
Images
  ↓
Email
  ↓
Redis
  ↓
WebSocket
```

Phase 8

```
Testing
  ↓
Docker
  ↓
GitHub
  ↓
CI/CD
  ↓
Cloud Deployment
```

Key Architecture — the one thing to remember

```
React
  |
  | REST API
```



Redis, WebSocket, payment integration, cloud storage, Docker, and CI/CD are kept as later phases so the initial project does not become unnecessarily complicated.